RESEARCH MEMO

# Building a Research Agent That Remembers

## 3-Layer Memory + Token Budget Guard + n8n Routing

What I built, what I measured, and what I learned

Python ChromaDB Anthropic SDK FastAPI n8n

Darshil Shukla

# The Problem

Most LLM usage in research workflows is broken in 3 ways:

**No memory**

Every query starts from zero — no context, no history

**No cost control**

One complex prompt can cost $2–5 with no guardrails

**No audit trail**

You can't prove what was asked, what it cost, or how good the answer was

# The Architecture

A 5-layer pipeline where every layer has one job

**Router + Budget Guard**
Hard token cap enforced before every API call

**Memory Layer**
Vector RAG + Episodic Buffer + Summary Cascade

**Context Assembler**
Greedy bin-packing — fits memory into remaining budget

**Claude (tool use)**
Haiku for search/decompose | Opus for final synthesis

**Evaluation Logger**
Tokens, cost, strategies, self-score → evaluation.md

# The 3-Layer Memory System

### Vector RAG

**Long-term**

ChromaDB semantic search — persists across sessions.
Every answer gets stored back as future knowledge.

### Episodic Buffer

**Short-term**

Last 10 turns of the current session in RAM.
Prevents re-asking the same sub-questions.

### Summary Cascade

**Compressed**

When buffer fills, oldest 5 turns are compressed
into a rolling 200-word summary by Claude.

# Key Design Trade-offs

Every architecture choice had an explicit reason and an accepted cost

| DECISION | WHY | TRADE-OFF |
|---|---|---|
| **ChromaDB over Pinecone** | Zero infra, <5ms latency, free | Weaker embedding model |
| **Ring buffer over full history** | O(1) context cost per call | Old turns lost if cascade skipped |
| **Greedy packing over DP** | O(n) speed in the hot path | ~2% sub-optimal packing |
| **Haiku for tool calls** | 60-70% cost reduction | Slightly less reasoning depth |
| **tiktoken for estimation** | Pre-call budget check | +-2-5% vs Claude tokeniser |

# Findings — Real Run Data

Every run logged to evaluation.md — these are actual numbers

**$0.07**
cheapest successful run

**19.4%**
lowest budget utilisation

**5**
sub-questions on first RLHF query

| Query | Tokens | Cost | Util% |
|---|---|---|---|
| RLHF vs DPO | 9,713 | $0.11 | 19.4% |
| Transformer architecture | 13,370 | $0.11 | 89.1% |
| Evolution of AI | 13,620 | $0.11 | 27.2% |
| UCL season (web search) | 11,046 | $0.07 | 22.1% |

# The Model Split Strategy

Using two Claude models for different jobs cut cost by ~65%

### claude-haiku-4-5

**Fast + Cheap**

- Decompose questions
- Search knowledge base
- Search the web
- Self-score answers

$1 input / $5 output per 1M tokens

### claude-opus-4-6

**High Quality**

- Final answer synthesis
- Answer formatting + polish
- Summary cascade compression

$5 input / $25 output per 1M tokens

**Result:** ~65% cost reduction vs using Opus for everything, no quality drop on tool-use steps

# Key Learnings

What surprised me building this

**Token counting is not the same as token charging**

tiktoken estimates diverge 2-5% from Claude's actual count. You need a safety margin.

**Memory strategy matters more than model quality**

A well-assembled 8k context with the right chunks beats a 50k dump of everything.

**Self-scoring is surprisingly honest**

Haiku consistently scored knowledge-gap answers lower (3/5) and substantive answers higher (4-5/5).

**n8n routing pays off fast**

Simple queries auto-routed to 15k budget saved ~$0.03 each. At scale that compounds quickly.

**ChromaDB is underrated for local RAG**

Sub-5ms retrieval, zero infra, persistent across restarts. Only limitation is embedding quality.

OPEN SOURCE

# Everything is on GitHub. Clone it, break it, improve it.

github.com/DarshilShukla26/deep-research-agent

Python ChromaDB Anthropic SDK FastAPI n8n DuckDuckGo Search

If you're working on memory systems, cost-aware agents, or n8n automation — I'd love to connect.

**Darshil Shukla**
Graduate AI Engineer